Where Is My Route? Enabling Source Routing in LEO Satellite Networks

Satvik Vemuganti and Anand M. Baswade
Department of Computer Science and Engineering, Indian Institute of Technology Bhilai, India
Email: {vemugantisesha, anand}@iitbhilai.ac.in

2024 IEEE 99th Vehicular Technology Conference (VTC2024-Spring) | 979-8-3503-8741-4/24/$31.00 ©2024 IEEE | DOI: 10.1109/VTC2024-SPRING62846.2024.10683192

***Abstract*—A combination of decreasing rocket payload costs and discovery of cutting-edge applications of satellites in low earth orbit has brought about a generation of "New Space" start-ups looking to cash-in on the opportunity. One such application is to provide low-latency internet to the most remote regions of the world. Companies have deployed thousands of, and are looking to deploy tens of thousands more, satellites to Low Earth Orbit. The low altitudes and wide expanse of these mega-constellations help keep latency to a minimum while providing internet coverage to regions that are inaccessible to terrestrial networks. However, the highly dynamic nature of these constellations presents the problem of internet packet routing, which is well addressed in conventional terrestrial networks. We show that the number of changes in topology from second to second can be in the order of a few hundreds and that each change takes 70-80 time slots to be reflected in forwarding tables of 90% of the network. As an alternative, we present a source routing solution coupled with a packet carried forwarding state that addresses all the above issues. We also provide a heuristic to computing network topology that can potentially speed up topology computation by 3×. Our experiments show that source routing is feasible with manageable overhead and is scalable to thousands of routing nodes.**

***Index Terms*—source-routing, packet-carried-forwarding-state, LEO-SNs, Intra-Domain**

I. Introduction

Payload costs for space launch have decreased from tens of thousands of dollars per kilogram in NASA's space shuttle in the 1980s to a sub-thousand dollar per KG cost achieved by SpacEx's Falcon Heavy in 2020, and expected to drop even further. Rapid technological advancements enabling the reusability of launch rockets have had profound domino effects on research in domains ranging from disaster response to in-orbit computing applications [1]. Such low launch costs have made it economically feasible for players to deploy satellite networks to offer yet another application, low latency internet [2].

Concurrently, satellites in low earth orbit are highly dynamic, orbiting the earth at a frequency of once every 100 minutes. Such a high level of topology dynamism is a potential roadblock to every application being proposed for Low Earth Orbit Satellite Networks (LEO-SNs). Disaster response applications require data to be sent in real time from locations of disaster to locations that can effectively trigger disaster response. Caching applications require an abstraction preserving spatial and temporal locality of data. Satellite Internet Service Providers (ISPs) require data to be routed as efficiently as possible to provide ultra low latencies to the end users. The variability of paths in LEO-SNs is formally characterized in [3]. The work also models the frequent rerouting (route churn) caused due to paths getting created and destroyed. Each application thus calls for a novel mechanism to route data packets that considers issues presented by such dynamic networks.

As demonstrated in [3], conventional distributed routing protocols perform poorly mainly due to the dynamic topology on hand. While the source routing paradigm is not novel, most prior work studied source routing in scenarios where each user device is a node. However, in our case, while the number of satellites in the network are estimated to be in the order of the tens of thousands, the number of Ground Stations (GSTs) shall remain in the order of hundreds. Additionally, even though the satellite networks are highly dynamic, their trajectories are predictable thanks to well developed perturbation models [4]. An entirely centralized routing mechanism as presented in [5] would be extremely inefficient as the burden of route computation and topology maintenance rests at a single point. Computed information then has to be updated throughout the network.

A. Related Work

The cost–performance trade-offs in the design space for Internet routing that incorporates satellite connectivity are studied in [2]. It examines four solutions including a clean slate approach based on the SCION internet architecture [6] and an extension based on GSTs placed at Points of Presence (PoPs) in today's internet.

Due to the less flexibility, planning-intensive, and predictable nature of satellite networks, several SDN approaches have been proposed and strsudied extensively. SDR-Algorithm for NTN [7] and fybrrLink [5] are SDN approaches with the latter presenting a QoS-aware routing algorithm for satellite networks which computes optimal paths with a significant improvement in computation time when compared to other state-of-the-art routing algorithms for satellite networks.

There is also a family of link state routing protocols for intradomain routing in LEO-SNs. Taking advantage of the regularity of the constellation, [8] conducts periodic route calculation for instantaneous routing table generation, which well handles the regular topology changes.

Multilayer Satellite Routing algorithm [9], Satellite Grouping and Routing Protocol (SGRP) [10], Hierarchical Satellite

Routing Protocol [11], SAR-Algorithm [12], data-based inter-layer routing mechanism [13], and Expanding Range Route Selection [14] are all algorithms for routing in multi layer satellite networks. Our routing strategy assumes a LEO constellation, along with 3 GEO satellites.

In [15], authors address issues due to unreliability of routing path with a time-varying topology model for dynamic routing in LEO-SNs. [16] proposes a secure routing algorithm to address privacy issues arising from instability, openness and exposure of inter-satellite links. [17] provides an energy-efficient routing algorithm for internet packets, addressing the energy limitation of compute on satellites. However, they perform expensive route calculation on the satellites. ASER [18] provides an innovative approach to routing by dividing the constellation into clusters of satellites based on geographic regions. However, the authors developed ASER based on OSPF, involving forwarding tables at intermediate satellites, bringing with it all the issues associated with distributed routing protocols. [19] is the most relevant work that addresses source routing in LEO-SNs. However, it's major drawback is that it performs expensive route computation on the satellite, which has limited compute power, and does not account for the scale that LEO constellations are taking today.

B. Contributions

In this paper, we explore the possibility of a partially decentralized routing scheme. We begin by defining our system model at two levels of abstraction. A broader view of the network consists of a network of GSTs, LEO satellites and Geostationary (GEO) satellites as the service provider's infrastructure. Here are our key contributions:

- We study why conventional distributed routing protocols perform far from optimal in case of highly dynamic LEO-SNs.
- We present a heuristic that potentially reduces topology construction time by more than $3\times$.
- We introduce source routing as a potential solution for routing internet packets in LEO-SNs.
- We demonstrate the need of a Packet Carried Forwarding State based approach to be coupled with source routing for optimal performance.

II. Problem Motivation

We attempt to understand why the need for a novel or rethought routing mechanism arises in case of LEO-SNs. The basic fact on hand is that satellites in the LEO range are highly dynamic, with each satellite orbiting the earth once every 100 minutes. This results in a large number of changes in the topology very frequently. Existing internet data routing protocols were designed to be able to handle links that fail for any number of reasons. It is important to note that these changes due to link failures are of the order of a small fraction of the number of routing nodes in the network. However, this does not hold true for LEO-SNs. We observe the consequences of this through two experiments using simulation parameters given in Table I.

First, we construct the graph of a satellite network at a certain time. We assume this time to be $t = 0$. We then compute the graph at intervals of 1 second. Finally, we count the number of changes in the topology from the current graph to the graph of the preceding second. The changes count is the number of links that have been created or destroyed as compared to the previous graph. To understand the variation in number of changes better, we separate the data minute-wise. The number of changes per second across a minute is plotted across 9 minutes in Figure 1. We observe that the number of changes remains within the range of 350 to 500 per second across 9 minutes. If the topology changes so much each second, using a conventional, distributed routing protocol would congest the network with control messages.

TABLE I: Simulation Parameters

Parameter	**Value**
Simulated Constellation	Starlink (SpaceX)
Number of Satellites	4672
Inclinations of orbits	53.0^o
Simulation duration	9 minutes
Simulation interval	1 second
Satellite Orbit Period	100 minutes
Number of GSTs	626

This brings us to the second reason why distributed routing protocols would fail in such a network. In a distributed routing setting, a node updates it's routing tables when it receives information about the network from one of it's neighbours. This would mean that a change in one corner of the network topology, would take a very long time to get reflected in the routing tables of all routing nodes in the network. We examine this situation in Figure 2.

We consider a change in one corner of the network, particularly, a link becoming invalid. A time slot is the period of time it takes for a node to inform all of it's neighbours about the latest update in the network it has received. And so initially only 2 nodes are aware of the invalid link - the nodes between which the link was previously existing. This information propagates throughout the network as a 2-starting node Breadth First Search. We observe that it takes **76** time slots for information about a single change to propagate to 90% of the network. This causes packets to be incorrectly routed, resulting in a large number of packets being dropped.

III. System Model

The system model used in this paper is described at two levels. The first level can be considered as the network infrastructure deployed by the Satellite Internet Service Provider. It consists of an expansive network of LEO Satellites acting as routing nodes. Satellites may be deployed in polar or inclined orbits. Inclined orbits are generally preferred due to the dense population along them. Figure 3 illustrates a pair of intersecting inclined orbits of LEO Satellites from the constellation.

The ISP also deploys a constellation of LEO satellites. For the purposes of this work we consider SpaceX's Starlink constellation of 4672 satellites. Each LEO satellite has inter satellite links (ISLs) with 4 adjacent satellites; 2 of which are

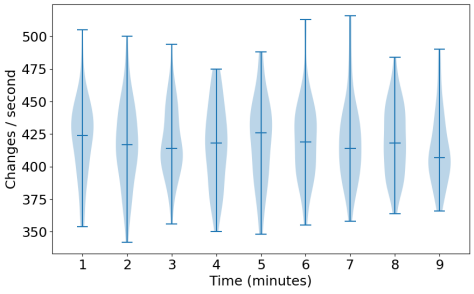

Fig. 1: Violin plot showing number of changes in graph over time. The number of changes in the network topology from second to second are calculated. This number, over a minute, is plotted as a violin. This is done across 9 minutes.

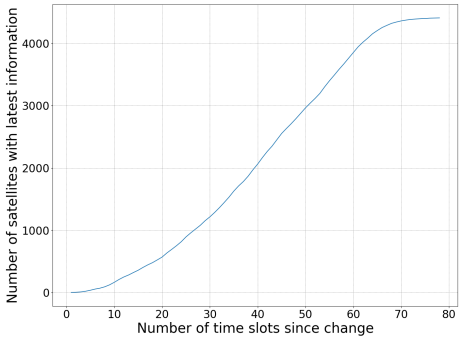

Fig. 2: The propagation of an update in a link is depicted in this figure. When a link gets created or destroyed, the information is passed on from each node aware of the information to each of its neighbors in a time slot and so on. It takes 76 time slots for link update information to propagate to 90% of the network.

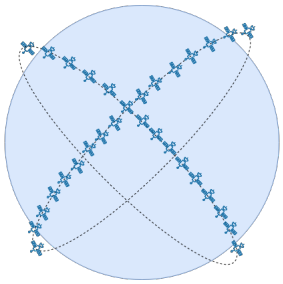

Fig. 3: Broad system model depicting two intersecting orbits of Low Earth Orbit satellites. The entire constellation consists of several inclined orbits of LEO Satellites.

within the same orbit (before and after) and the other 2 are with the closest 2 satellites in the entire constellation. Finally, the ISP also deploys a network of GSTs. As suggested in [2], these GSTs can be considered to be present at internet exchange points (IXPs) across the world. We consider a deployment consisting of 626 GSTs at IXPs around the world [20].

The second level of the system model is referred to as the user-centric view of the network. In this view, there exists a pair of end users as shown in Figure 4. Currently, SpaceX supplies it's customers with a Pizza-box sized dish that can communicate with LEO satellites. However, this is not shown in Figure 4 as it is equivalent to an end device directly communicating with the satellites.

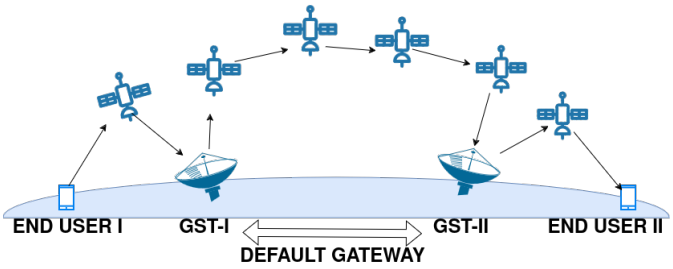

Fig. 4: A user centric view of the network depicting end mobile devices, a first hop satellite acting as a repeater and the GST as a default gateway to the LEO-SN.

All communication from the end user passes through a GST. Particularly, a GST acts as the default gateway for each data packet from an end user. This means that all routing decisions for the data packet can be made at the default gateway. Once a GST receives a data packet, it computes the route to the destination GST for the received packet, embeds the path in the packet and forwards the packet over the LEO-SN. The GST maintains a routing table with paths to rest of the GSTs in the network.

IV. Topology-Graph Construction

In order to perform source routing at each of the gateway GSTs, the topology of the network needs to be calculated efficiently. This is not as straightforward as it seems. First, we need to obtain the position of each satellite at any point in time. This can be done using satellite propagation libraries available in Python. We use the Python SGP4 library [21] for this. We are able to obtain the predicted positions of each and every satellite in Earth-Centered Inertial (ECI) Coordinates. This coordinate system is convenient in order to calculate the distance between a pair of satellites. We compute the distance between every pair of satellites and for every satellite choose the 4 nearest satellites to be it's neighbours in the graph. This way, the topology graph can be computed easily every second.

However, the above method is inefficient, as it computes the distance between all pairs of satellites in order to determine which satellites share an inter-satellite link. We propose a heuristic that can reduce the search space of potential neighbours for each satellite. We first calculate the coordinates of every satellite at a chosen time instance in the Geodesic Coordinate System using the ECI coordinates obtained earlier. At this point we define a variable called *threshold* whose unit is *degree*.

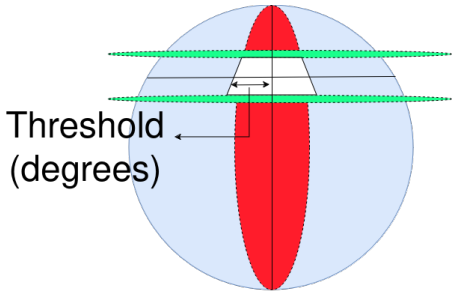

Fig. 5: Assume that a satellite is present at the intersection of the lines of latitude and longitude shown in the figure. We choose a region enclosed by a pair of latitude planes and a pair of longitudinal planes. We only compute distances between the satellite and other satellites that are enclosed in this region to determine which 4 satellites a link shall be established with. By default, 2 satellites will always happen to be the immediately preceding and succeeding satellite in the same orbit.

Varying the threshold from 1 to 36, for each satellite in the constellation, we obtain 4 values from its current geodetic coordinates and threshold as follows,

$$minimum\_latitude = latitude - threshold$$

$$maximum\_latitude = latitude + threshold$$

$$minimum\_longitude = longitude - threshold$$

$$maximum\_longitude = longitude + threshold$$

The region bounded by the above minimum and maximum latitudes and longitudes is depicted in Figure 5. For each satellite, we count the number of satellites of the constellation that lie within the range of minimum and maximum latitudes and longitudes. The counts are plotted in Figure 6 as a series of box plots for each threshold from 1 to 36. It can be observed that for a very big range of thresholds, the minimum number of neighboring satellites in the region remains 0. This can be attributed to the fact that at any point in time, some satellites are isolated from the rest of the constellation because of inclined orbits. It can be observed that the minimum value is above the red line ($y = 5$) at a threshold of 31. In spite of our system model specifying that each satellite will have only 4 neighbours, we are taking a safe count as 5 to pick the optimal threshold.

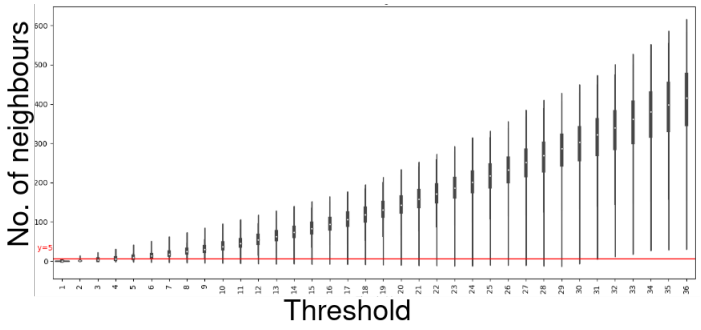

Fig. 6: Graph showing variation of number of neighbours in regions of varying thresholds. At each threshold on the x-axis, there is a box plot on with the count of number of neighbours on the y-axis.

Even if we choose such a high threshold *i.e.,* 31, to account for different patterns of constellations that may be deployed in the coming years, the maximum number of satellite in the region remains under 500. This is a more than **7**$\times$ reduction in search space. To observe the difference in topology construction between an all pairs comparison and a reduced search space approach, we noted the graph construction times for a topology every second over 10 minutes across thresholds of 30, 35, 40, and 45. As shown in Figure 7, there is more than **3**$\times$ improvement in graph computation time with the help of the proposed heuristic if we choose a threshold of 30.

V. Source Routing

Once we have computed the topology of the network, we have the option to choose between a centralized or decentralized routing scheme. In a centralized scheme, all paths are computed at one point in the network and updated in the routing tables of all intermediate nodes. A comprehensive analysis of centralized routing in the context of LEO-SNs has been performed in [5] where the routes are updated as flow rules at the intermediate satellites in a timely manner. However, a major flaw of such an approach is scalability. It becomes highly inefficient to compute routes for the entire

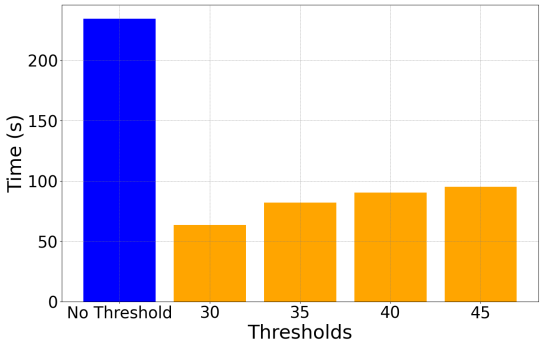

Fig. 7: Plot comparing the average topology computation times at various thresholds. Thresholds 30^o, 35^o, 40^o, and 45^o are shown in orange whereas the computation time without any filtering heuristic is shown in the blue bar.

network infrastructure from a single point. On the other hand, in a decentralized approach, each node obtains information about the topology from neighbouring nodes to update it's local routing tables. All forwarding decisions are then taken based on these routing tables. As demonstrated in Section II, the decentralized approach fails in a highly dynamic setting as in the case of LEO-SNs.

So we attempt to observe if there is any middle ground that can be occupied by an optimal routing scheme. It would be prudent to observe exactly how much the route computation time scales as the number of nodes in the network increases. So we design a simulation to observe this value. We vary the number of nodes in our constructed topology from 500 to 4,500 at steps of 500 and sample a set of 100,000 (source, destination) pairs in the network. At each step, we compute the median and maximum route computation times. The plot in Figure 8 demonstrates how the route computation time scales as the number of nodes in the network increases. Note that the we have a linearly scaling curve and not an exponential curve.

From the last plot, we can conclude that although a centralized source routing solution is not feasible, having multiple nodes performing source routing, *i.e.,* decentralized source routing, might be a scalable option. This is exactly the solution space we are targeting. In our system model, the service provider installs GSTs which connect to the LEO-SN through GST Satellite Links (GSLs). Every packet that the end user sends is directed to the nearest GST (acting as a gateway) through the forst hop LEO satellite. Each GST maintains a routing table which contains routes to every GST in the network. This table will be of size in the order of hundreds as compared to the constellation sizes being planned to be of the order of a few thousands. When idle, each GST computes paths to other GSTs in the network. However, the question of the overhead of source routing is not addressed completely yet. So we calculate the overhead of route computation time in end-to-end latency.

We compute the average route computation time from the GST to a set of 626 arbitrarily placed GSTs around the world [2]. We then calculate the propagation delay of packets along each of the computed paths. Finally, we plot the percentage of the total time consumed by the route computation component

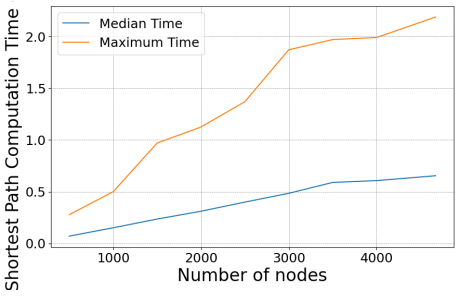

Fig. 8: This figure depicts scalability of source routing. As the number of routing nodes in the network increases, the route computation time does not grow exponentially.

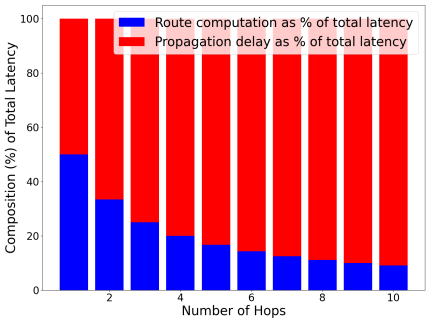

Fig. 9: Comparison between route computation time and propagation delay as the number of hops on the computed path increases.

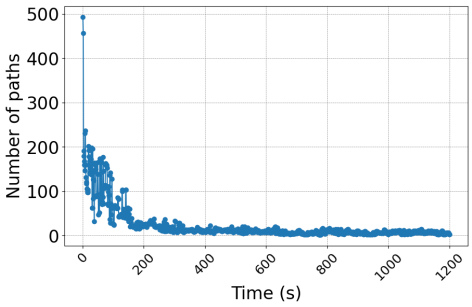

Fig. 10: The number of paths that were valid in the previous second and continue to remain valid in the topology computed at the current second are plotted on the y-axis.

and propagation component in separate colors. A striking observation can be made if these are plotted against path lengths on the x-axis as in Figure 9. For paths with very few hops in the satellite network, the route computation overhead is a huge fraction of the total latency. However, for paths with many hops, the route computation overhead is a very small fraction of the total latency. With the processing power available on computers these days, the route computation time is practically constant when varying the number of hops. On the other hand, propagation delay increases greatly as the number of hops increases. Thus we observe that as the number of hops increases, route computation time decreases as a fraction of total latency. This observation supports the argument for feasibility of a distributed source routing solution.

VI. Packet Carried Forwarding State

Once a path has been computed using the source routing techniques described, flow rules can be propagated to every routing satellite in the network. However, updating paths using flow rules can be consequential to the source routing mechanism being proposed. Due to the large number of changes in the network topology from second to second, several paths become invalid very rapidly. There is a rapid degradation in path validity and quality in LEO-SNs as extensively demonstrated in [3]. We demonstrate an example of rapidly degrading paths in Figure 10.

The y-axis contains the value of number of valid paths and the x-axis contains the progression of time in seconds from 0 to 1200 seconds. We first create a set of paths that are valid in the topology at $t = 0$. Then at each subsequent second's topology we count how many of the originally valid paths are currently valid. We distinctly observe the degradation in this plot. Within the first 3 minutes, more than 80% of the originally valid paths become invalid. Additionally, this plot provides an interesting insight into the dynamic nature of the topology. While some paths get destroyed between subsequent seconds, they might become briefly valid due to the symmetry of the constellation before becoming invalid permanently again. This behaviour causes the seemingly irregular jumps in a plot which is supposed to exclusively decrease.

And so we propose the alternative approach of embedding the path in the packet itself. The benefits of this approach are straightforward from the discussion above. The rapid degradation in valid paths does not affect our approach if the paths being calculated in real time on the latest topology graph is embedded in the packet itself. The only drawback of this approach is the extra data overhead in embedding each packet with the path. However, this overhead is manageable for the network infrastructure, and by far outweighs the consequences of high packet drop rates as explained below.

Overhead. We anticipate that there will be tens of thousands of satellites in orbit in a full size constellation. Satellite identifiers can be 2 Bytes in length. However, to account for unexpected growth, we allow 3 Bytes for satellite identifiers. On all of the constellation graphs that we computed, we ran a breadth-first search to obtain the diameter and observed it to be 14 nodes (13 links). This means that if we were to embed the path in every network packet, it would be a 42 Byte overhead per packet, which is not justifiable. Instead, we observe that every satellite has at most 4 inter-satellite links (ISLs), i.e. just 2-bits of information. So each packet can have at-most 3 Bytes (source satellite identifier) + 26-bits (13 intermediate satellites) + 3Bytes (destination satellite identifier) = 10 Bytes (Rounded up). In comparison, the IP-header is from 20-60 Bytes. This shows that the data overhead of Packet Carried Forwarding State (PCFS) is not too high.

Packet Drop Rates. Traditional routing protocols consist of link update messages. In the case of LEO-SNs, link updates need to happen very frequently. However, it happens to be that by the time an update in a link is shared with every satellite in the network, the routes involving that link are no longer valid. On the contrary, since trajectories of satellites can be predicted, the graph of the network can be calculated ahead of time. A route is computed upon this latest network-graph and embedded in each packet being transmitted. This helps avoid dropping of packets due to routes becoming invalid. Packet drop rates can be correlated with the plot showing how routes become invalid.

VII. Discussion and Conclusion

Source routing paves the way for network traffic engineering. Particularly, competing Satellite Network Internet Service Providers (SN-ISPs) could establish inter satellite links across satellites from different constellations. Economic modelling of feasibility and benefits for either of the parties would help decide which party gains more from the transaction.

The notion of "anywhere compute" introduced in [1] can be leveraged to any number of interesting applications ranging from space Content Delivery Networks (CDN) to "in the cloud compute".

In this work, we first provided two reasons why conventional distributed routing protocols will fail in a highly dynamic network such as LEO satellite constellations - too many changes in the network topology from second to second and the long duration it takes for a link update to propagate throughout the network. We then define a system model through two views; the service provider and end-user point of view. We provide and analyse a heuristic to efficiently compute the network topology at any point in time using existing perturbation models on two-line element files. We present a distributed source routing scheme to work around the shortcomings of existing protocols. Finally, we argue for the use of a PCFS as compared to updating flow rules at intermediate nodes.

REFERENCES

[1] D. Bhattacherjee, S. Kassing, M. Licciardello, and A. Singla, "In-orbit computing: An outlandish thought experiment?" in *Proc. of the 19th ACM Workshop on Hot Topics in Networks*, 2020, pp. 197–204.

[2] G. Giuliari, T. Klenze, M. Legner, D. Basin, A. Perrig, and A. Singla, "Internet backbones in space," *ACM SIGCOMM Computer Communication Review*, vol. 50, no. 1, pp. 25–37, 2020.

[3] V. Bhosale, A. Saeed, K. Bhardwaj, and A. Gavrilovska, "A characterization of route variability in leo satellite networks," in *Passive and Active Measurement*, A. Brunstrom, M. Flores, and M. Fiore, Eds. Cham: Springer Nature Switzerland, 2023, pp. 313–342.

[4] D. Vallado, P. Crawford, R. Hujsak, and T. Kelso, *Revisiting Spacetrack Report #3*.

[5] P. Kumar, S. Bhushan, D. Halder, and A. M. Baswade, "fybrrlink: Efficient qos-aware routing in sdn enabled future satellite networks," *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, pp. 2107–2118, 2021.

[6] X. Zhang, H.-C. Hsiao, G. Hasker, H. Chan, A. Perrig, and D. G. Andersen, "Scion: Scalability, control, and isolation on next-generation networks," in *Proc. of the IEEE Symposium on Security and Privacy*. IEEE, 2011, pp. 212–227.

[7] Y. Zhu, L. Qian, L. Ding, F. Yang, C. Zhi, and T. Song, "Software defined routing algorithm in leo satellite networks," in *Proc. of the International Conference on Electrical Engineering and Informatics (ICELTICs)*. IEEE, 2017, pp. 257–262.

[8] L. Zhang, F. Yan, Y. Zhang, T. Wu, Y. Zhu, W. Xia, and L. Shen, "A routing algorithm based on link state information for leo satellite networks," in *Proc. of the IEEE Globecom Workshops*. IEEE, 2020, pp. 1–6.

[9] I. F. Akyildiz, E. Ekici, and M. D. Bender, "Mlsr: A novel routing algorithm for multilayered satellite ip networks," *IEEE/ACM Transactions on networking*, vol. 10, no. 3, pp. 411–424, 2002.

[10] C. Chen and E. Ekici, "A routing protocol for hierarchical leo/meo satellite ip networks," *Wireless networks*, vol. 11, pp. 507–521, 2005.

[11] J.-W. Lee, J.-W. Lee, T.-W. Kim, and D.-U. Kim, "Satellite over satellite (sos) network: a novel concept of hierarchical architecture and routing in satellite network," in *Proc. of the 25th Annual IEEE Conference on Local Computer Networks.* IEEE, 2000, pp. 392–399.

[12] L. Jun, Z. Ji-Wei, and X. Nan, "Research and simulation on an autonomous routing algorithm for geo_leo satellite networks," in *Proc. of the Fourth International Conference on Intelligent Computation Technology and Automation*, vol. 2. IEEE, 2011, pp. 657–660.

[13] X. Feng, M. Yang, and Q. Guo, "A novel distributed routing algorithm based on data-driven in geo/leo hybrid satellite network," in *Proc. of the International Conference on Wireless Communications & Signal Processing (WCSP)*. IEEE, 2015, pp. 1–5.

[14] Y. Wu, Z. Yang, and Q. Zhang, "A novel dtn routing algorithm in the geo-relaying satellite network," in *Proc. of the 11th International Conference on Mobile Ad-hoc and Sensor Networks (MSN)*. IEEE, 2015, pp. 264–269.

[15] Z. Han, C. Xu, G. Zhao, S. Wang, K. Cheng, and S. Yu, "Time-varying topology model for dynamic routing in leo satellite constellation networks," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 3, pp. 3440–3454, 2023.

[16] H. Li, D. Shi, W. Wang, D. Liao, T. R. Gadekallu, and K. Yu, "Secure routing for leo satellite network survivability," *Computer Networks*, vol. 211, p. 109011, 2022.

[17] R. do N. Mota Macambira, C. B. Carvalho, and J. F. de Rezende, "Energy-efficient routing in leo satellite networks for extending satellites lifetime," *Computer Communications*, vol. 195, pp. 463–475, 2022.

[18] X. Zhang, Y. Yang, M. Xu, and J. Luo, "Aser: Scalable distributed routing protocol for leo satellite networks," in *Proc. of the 46th IEEE Conference on Local Computer Networks (LCN)*. IEEE, 2021, pp. 65–72.

[19] X. Song, K. Liu, J. Zhang, and L. Cheng, "A source routing algorithm for leo satellite networks," in *2005 IEEE International Symposium on Microwave, Antenna, Propagation and EMC Technologies for Wireless Communications*, vol. 2, 2005, pp. 1351–1355 Vol. 2.

[20] F. Wohlfart, N. Chatzis, C. Dabanoglu, G. Carle, and W. Willinger, "Leveraging interconnections for performance: the serving infrastructure of a large cdn," in *Proc. of the Conference of the ACM Special Interest Group on Data Communication*, 2018, pp. 206–220.

[21] B. Rhodes, D. B. Vallado, and T. Magakalang, "sgp4: Python implementation of the SGP4 satellite orbit propagation model," https://pypi.org/project/sgp4/, 2023.